

中山大學
SUN YAT-SEN UNIVERSITY

数字图像处理—第六章作业

彩色图像直方图均衡化实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 骆伟祺

起 始 日 期: 2025年12月3日

结 束 日 期: 2025年12月3日

实 验 地 点: B205

目录

1 实验简述 **2**

2 理论知识 **2**

2.1 HSI色彩空间处理 2

2.2 RGB色彩空间处理 2

3 代码实现 **2**

4 结果分析 **4**

5 实验总结 **4**

5.1 收获 4

5.2 问题与解决 4

1 实验简述

本次实验要求对彩色图像进行直方图均衡化处理，并对比分析两种不同方法的处理效果：

1. **方法#1**：采用HSI空间，仅在I（强度）分量上作直方图均衡。
2. **方法#2**：采用RGB空间，分别在R、G、B三个分量上作直方图均衡。

实验旨在探究不同色彩空间处理对图像色彩保真度和对比度的影响。

2 理论知识

2.1 HSI色彩空间处理

HSI（Hue, Saturation, Intensity）模型反映了人眼感知颜色的方式。在图像增强中，通常希望保持图像的色调（Hue）和饱和度（Saturation）不变，仅调整亮度（Intensity）。

- **原理**：将RGB图像转换为HSI图像，提取I分量进行直方图均衡化得到I'，保持H和S不变，最后逆变换回RGB空间。
- **简化计算**：由于仅需调整亮度，可利用比例关系 $R' = R \cdot (I'/I)$（G、B同理）直接在RGB空间操作，避免复杂的空间转换。

2.2 RGB色彩空间处理

- **原理**：直接对R、G、B三个通道分别进行直方图均衡化。
- **缺陷**：直方图均衡化是非线性变换。分别处理三个通道会改变像素点R、G、B分量之间的比例关系，从而导致合成颜色的色调和饱和度发生改变，产生颜色失真。

3 代码实现

实验核心代码如下：

```
def bgr_to_hsi(img):
    img_float = img.astype(np.float32) / 255.0
    b, g, r = cv2.split(img_float)

    # 计算分子和分母，得到theta
    num = 0.5 * ((r - g) + (r - b))
    den = np.sqrt((r - g) ** 2 + (r - b) * (g - b))
    theta = np.arccos(np.clip(num / (den + 1e-6), -1, 1))  # 防止除零

    # 计算分量H
    h = theta.copy()
    h[b > g] = 2 * np.pi - h[b > g]

    # 计算分量S
    sum_rgb = r + g + b
    min_rgb = np.minimum(np.minimum(r, g), b)
    s = 1 - 3 * min_rgb / (sum_rgb + 1e-6)
    s[sum_rgb == 0] = 0
```

```
    # 计算分量I
    i = sum_rgb / 3.0
    return cv2.merge((h, s, i))

def hsi_to_bgr(img_hsi):
    h, s, i = cv2.split(img_hsi)
    b, g, r = np.zeros_like(h), np.zeros_like(h), np.zeros_like(h)
    epsilon = 1e-6  # 防止除零

    # 区间0: 0 <= H < 度120
    mask1 = (h < 2 * np.pi / 3)
    h1 = h[mask1]
    b[mask1] = i[mask1] * (1 - s[mask1])
    r[mask1] = i[mask1] * (1 + (s[mask1] * np.cos(h1)) / (np.cos(np.pi / 3 - h1) + epsilon
        ))
    g[mask1] = 3 * i[mask1] - (r[mask1] + b[mask1])

    # 区间1: 120 <= H < 度240
    mask2 = (h >= 2 * np.pi / 3) & (h < 4 * np.pi / 3)
    h2 = h[mask2] - 2 * np.pi / 3
    r[mask2] = i[mask2] * (1 - s[mask2])
    g[mask2] = i[mask2] * (1 + (s[mask2] * np.cos(h2)) / (np.cos(np.pi / 3 - h2) + epsilon
        ))
    b[mask2] = 3 * i[mask2] - (r[mask2] + g[mask2])

    # 区间2: 240 <= H < 度360
    mask3 = (h >= 4 * np.pi / 3)
    h3 = h[mask3] - 4 * np.pi / 3
    g[mask3] = i[mask3] * (1 - s[mask3])
    b[mask3] = i[mask3] * (1 + (s[mask3] * np.cos(h3)) / (np.cos(np.pi / 3 - h3) + epsilon
        ))
    r[mask3] = 3 * i[mask3] - (g[mask3] + b[mask3])
    img_bgr = cv2.merge((b, g, r))
    img_bgr = np.clip(img_bgr, 0, 1)
    return np.clip(img_bgr * 255, 0, 255).astype(np.uint8)

def main():
    #...#
    # 方法:  #1HSI 空间均衡化 I 分量
    hsi_img = bgr_to_hsi(img)
    h, s, i = cv2.split(hsi_img)

    # 对强度 I 进行直方图均衡化
    i_uint8 = (i * 255).astype(np.uint8)
    i_eq_uint8 = cv2.equalizeHist(i_uint8)
    i_eq = i_eq_uint8.astype(np.float32) / 255.0

    # 合并通道并转回 BGR
    hsi_eq = cv2.merge((h, s, i_eq))
    img_hsi_eq = hsi_to_bgr(hsi_eq)

    # 方法 #2: RGB 空间分别均衡化
    b_raw, g_raw, r_raw = cv2.split(img)
    b_eq2 = cv2.equalizeHist(b_raw)
    g_eq2 = cv2.equalizeHist(g_raw)
    r_eq2 = cv2.equalizeHist(r_raw)
    img_rgb_eq = cv2.merge((b_eq2, g_eq2, r_eq2))
```

4　结果分析

实验结果如图1所示。



(a) 原始图像　(b) 方法1：HSI均衡化结果　(c) 方法2：RGB均衡化结果

图 1: 两种直方图均衡化方法对比

对比分析：

1. **方法1 (HSI)**：观察图1(b)，图像的整体亮度分布更加均匀，暗部细节得到提升。最重要的是，图像的色彩看起来非常自然，与原图(a)相比，色调没有发生明显偏移。这是因为该方法仅改变了强度分量，严格保持了色调和饱和度信息。

2. **方法2 (RGB)**：观察图1(c)，虽然图像对比度也得到了增强，但颜色出现了明显的怪异感（例如玻璃瓶液体的颜色变得不自然）。这是因为分别对三个通道进行非线性变换破坏了原始的色彩比例，导致了严重的色调偏移（Color Shift）。

5　实验总结

5.1　收获

- 理解了在彩色图像处理中，选择合适的色彩空间的重要性。RGB空间适合显示，而HSI等空间更适合图像分析和增强。
- 掌握了彩色图像直方图均衡化的两种策略，并验证了仅在亮度通道进行均衡化是保持色彩保真度的正确方法。

5.2　问题与解决

在实现HSI色彩空间转换的过程中，遇到了数值计算稳定性和结果完整性的问题，具体如下：

1. **HSI转RGB时的分母可能为零**：在计算 R, G, B 分量时（例如 $r = i \cdot (1 + \frac{s \cdot \cos(h)}{\cos(\pi/3 - h)})$），分母中的余弦项可能为零，导致除零错误。 **解决**：在分母加上一个极小值（如 10^{-6}）防止除零。

2. **HSI转RGB后数值范围溢出**：转换后的 R, G, B 值可能超出 $[0, 1]$ 范围，仅靠最后的类型转换可能不够。 **解决**：在计算分支后立即对 R, G, B 分量进行裁剪（`np.clip`），确保值在 $[0, 1]$ 内。

3. **验证色彩空间转换的正确性**：手动实现的转换公式较为复杂，容易出错。 **解决**：在进行主要处理之前，添加了一个简单的往返测试（RGB $\rightarrow$ HSI $\rightarrow$ RGB），计算重构图像与原图的均方误差（MSE）。如果 MSE 过大则发出警告，以此验证转换算法的准确性。